Implementing and Testing a Probing-Based Path Verification for PolKA Source Routing Protocol

Henrique C. Layber, Roberta L. Gomes, Magnos Martinello

[1]Department of Informatics
Federal University of Espírito Santo (UFES) – Vitória, ES – Brazil

henrique.layber@edu.ufes.br,{roberta.gomes, magnos.martinello}@ufes.br

Abstract. *Presents an implementation of a route probing and validation mechanism for the source routing protocol PolKA. The mechanism is based on a composition of checksum functions on stateless core switches, which allows a trusted party to verify if a packet traversed the network along the source-defined path.*

Resumo. *Este trabalho apresenta uma implementação de um mecanismo de sondagem e verificação de rota para um protocolo de roteamento na origem chamado PolKA. O mecanismo é baseado em uma combinação de funções de hash, calculadas por switches de núcleo sem estado, permitindo a uma parte confiável verificar se um pacote percorreu a rede ao longo do caminho definido pela origem.*

Checar se keywords estão boas para o SBC. Só copiei do **PathSec** **Keywords – Path Aware; Network Security; Path Verification; In-networking Programming**

1. Introduction

Ever since *Source Routing* (SR) was proposed, there has been a need to ensure that packets traverse the network along the paths selected by the source, not only for security reasons but also to ensure that the network is functioning correctly and correctly configured. This is particularly important in the context of *Software-Defined Networks* (SDNs), where the control plane can select paths based on a variety of criteria.

In this paper, we propose a new *P4* implementation for Probing-Based Path Verification (PBV), able to do verify the route used for a packet. This is achieved by using a composition of hash functions on stateless core switches. It can then be checked by a trusted party that knows the `node_id`s independently. This work is intended to implement part of **PathSec** and is available online on (`github.com/Henriquelay/polka-halfsiphash`).

2. Problem definition

A **PathSec** network consists of a logically centralized SDN Controller that selects routing paths and configures the nodes, edge nodes that embed metadata into packets, core nodes that execute basic packet forwarding with light signatures operations (proof-of-transit), and smart contracts running on a blockchain to support signatures verification and to register path compliance. A unique Residue Number System-based is used to generate a `route_id`, from which a core node calculates the next hop, using the node's secret `node_id` [Martinello et al. 2024]. For auditability, the resulting multi-signed would then be sent to a smart contract and blockchain system which are out of scope for this paper.

2.1. Problem formalization

Let i be the source node (**i**ngress node) and e be the destination node (**e**gress node). Let the path from i to e $P_{i \to e}$ be a sequence of nodes: the path is defined by $P_{i \to e} = (i, s_1, s_2, ..., s_{n-1}, s_n, e)$, s_n being the n-th core node in the path.

In SR protocols, the route up to the protocol boundary (usually, the SDN border) is defined in i. That is, i sets the packet header with enough information for each core node to calculate the next hop. Calculating each hop is done exploiting the Chinese Remainder Theorem[Dominicini et al. 2020], and is out of the scope of this paper.

The main problem we are trying to solve is path verification, that is, to have a way to ensure if the packet follows the path defined. A solution must be able to identify if the packet: (i) has passed through all nodes in the Path; (ii) has passed through the correct order of nodes; (iii) has **not** passed through any node that is **not** in the path.

More formally, given a sequence of nodes $P_{i \to e}$, and a captured sequence of nodes actually traversed P_j, a solution must identify if $P_{i \to e} = P_j$. Notably, it does not require validation, that is, it needs only to respond if $P_{i \to e} = P_j$ and does not need to know any of their contents or check their validity as a route.

2.2. Multi-signature model for Path Verification

To balance trace effectiveness and scalability, **PathSec** proposes using packets themselves as probes[Martinello et al. 2024][Basat 2020], with a lightweight multi-signature, keeping the header size fixed. That is, each core node hashes the previous' node signature with a key only itself (and the Controller) can possess, and pass it forward, and all following core nodes will do the same until the edge node is reached.

Está preciso e claro dizer que "which we will use to push information downstream"? Each node's execution plan is stateless. So, in terms of signature composition, a node n_i can be viewed as a function $f_{n_i}(x)$. A node can alter the header of the packet, which we will use to push information downstream and ultimately detect if the path taken is correct.

In order to represent all nodes by the same function, we assign a distinct key value k for each node n_i, and use a bivariate function $f(k_{n_i}, x) = f_{n_i}(x)$. By using functions in two variables and enforcing one of the variables to have any value that's unique between nodes, we ensure that the function's result is unique for each switch as long as the function is collision resistant enough, that is, $f_{n_a}(x) \neq f_{n_b}(x) \iff y \neq z$.

Using function composition appropriate to use, as it preserves the order-sensitive property of the path, since $f \circ g \neq g \circ f$ in a general case. In our model, each node will execute a single function of this composition, using the previous node's output as input. In this way, $(f_{n_1} \circ f_{n_2} \circ f_{n_3})(x) = f(k_{n_3}, f(k_{n_2}, f(k_{n_1}, x)))$, f being a collision-resistant function, preferably a hash function.

3. Implementing and Testing a Probing-Based Path Verification (PBV)

PBV consists of sending special packets (probes) with path-verification capabilities. The path verification system used is described in Section 2.2. Some assumptions are made: (i) Each node is assumed to be secure, that is, no node will alter the packet in any way that

is not expected. This is a common assumption in SDN networks, where a trusted party is the only entity that can alter the network state; (ii) Every link is assumed to be perfect, that is, no packet loss, no packet corruption, and no packet duplication; (iii) Protocol boundary is IPv4. This means that *PolKA* is only used inside this network, and only IPv4 is used outside; (iv) All paths are assumed to be valid and all information correct unless stated otherwise; (v) Hash collisions will not happen.

Comment

- explicação/motivação do uso do mininet , falando um pouco sobre p4 e bmv2 (ñn mencionar o wireshark aqui... pode mencionar na sec. de testes... como uma tool complementar)
- o mininet é inerentemetne isolado... mas o pathsec necessita de uma infraestrutura externa (blockchain).... motivação para usar o scapy
.... nas subseções a seguir são detalhados o setup do ambiente e a implementação

3.1. Setup

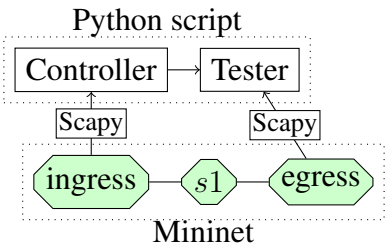

Figure 1. Experimentation setup

All experiments took place in a VM setup with Mininet-wifi[Fontes et al. 2015][1] and were targeting Mininet's[Lantz et al. 2010] Behavioral Model version 2[2](*BMv2*). Scapy[secdev 2003] was used to parse packets programmatically. All data plane code was written in *P4* [**?**]. Mininet-wifi was used instead of Mininet due to its libraries for deploying *P4 BMv2* switches.

3.2. Implementation

Rephrase By making the function f a hash function, and the unique identifier k_{s_i} as a composition including the `node_id`, we apply an input data (only external variable, seed) into a chain hash functions and verify if they match. For additional validation, we also integrate the calculated exit port into the checksum, covering some other forms of attacks or errors.

It was implemented as a version on PolKA, this means it uses the same `ethertype 0x1234` and is interoperable with PolKA. Up-to-date PolKA headers were used (and upgraded from the forked version) to ensure compatibility. It uses the `version` header field to differentiate between regular PolKA version packets and what we call *probe* packets. PolKA packets uses version `0x01`, and probe packets uses version `0xF1`. Figure 2 shows the used topology used in the experiments.

[1]Available on Mininet-wifi's repository `github.com/intrig-unicamp/mininet-wifi`
[2]`github.com/p4lang/behavioral-model`

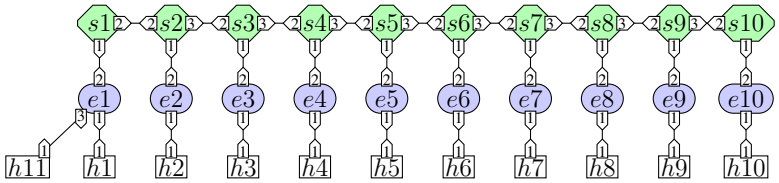

Figure 2. Baseline topology used in experimentation.h_i are hosts, e_i are edge nodes, s_i are core nodes

It reads as follows: s_1 is linked to s_2's port 2 using port 2, s_2 is linked to e_2's port 2 using port 1, and so on.

3.2.1. Parsing

Comment

FIGURA desses subsub, ilustrando esse ”pipeline”, explicar em tópicos

Parsing is done in edge nodes as follows:

- If an IPv4 protocol `ethertype` field is detected (`0x0800`), it must be a packet from outside the network, it must be wrapped and routed by the same edge node that parsed it. Let call this process be called *encapsulation*;
- If a PolKA protocol `ethertype` field is detected (`0x1234`), it must be a packet from inside the network, since the protocol boundary is IPv4, the original IPv4 packet must be unwrapped. Let this process be called *decapsulation*.

On core nodes, the packet is only parsed as PolKA packets, but it can be either a regular PolKA packet version or a probe packet version. If a probe packet version is detected, the probe packet header is parsed as well, otherwise the packet is treated a regular PolKA packet and no probing is done.

The implementation for parsing on edge nodes can be found in **??**, and for core nodes in **??**.

3.2.2. Encapsulation

PolKA headers consists of the route polynomial (`routeid`), along with `version`, `ttl` and `proto` (stores the original `ethertype`). `route_id` calculation is out of the scope of this paper.

A new header is added for probe packets, containing a 32 bit `key` and 32 bit `l_hash`.

During encapsulation of a probe packet, a random number is generated, and is used as `key`, for reproducibility and a seed for our composition. Edge nodes does not execute checksum functions and only repeats the key into the checksum field `l_hash`, so `node_id` for encapsulation is not required.

After encapsulation, the packet is sent to the next hop.

The implementation for encapsulation and related headers can be found in **??**.

3.2.3. Composition

Every core node does checksum trying to congregate the previous `l_hash`, the calculated next hop port, and its own `node_id` into the 32 bit field. Currently, it is implemented as such:

$$\texttt{l\_hash} \leftarrow \text{CRC32}(\text{exit port} \oplus \texttt{l\_hash} \oplus \texttt{node\_id})$$

The CRC32 checksum function used currently is the one available by *BMv2* standard library, and is experimentally found to be ISO HDLC[3].

3.2.4. Decapsulation

At the egress node, Polka headers are dropped, and the packet becomes an identical packet to what the ingress node received. The probe packet header is also dropped. The packet is then sent to the host. No checksum is calculated, so `node_id` is not required. Thus, together with 3.2.2, the `node_id` is not used for validation on edge nodes.

The implementation for decapsulation can be found in **??**.

The algorithm was verified externally through another program simulating all the composition steps, with source available[4], making use of the 'crc' library crate[5], and checked with gathered data.

The implementation for a node doing an individual checksum calculation step can be found in **??**.

A simple example of a packet traversing a network with 10 core switches is shown in the figure below. Exit port is calculated by PolKA. In the examples, we are measuring $P_{e_1 \to e_{10}} = (e_1, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9, s_{10}, s_{11}, e_{10})$

Comment

FIGURA

4. Adversity scenarios

Comment

- explica no 1o paragraf. que nesta sec. vc vai apresentar os testes representando diferente cenenários de falha/ataque
- CRIAR UMA FIGURONA, colocando as 4 fig juntas (faz um sub tit de fig (a), (b), etc)

[3] `reveng.sourceforge.io/crc-catalogue/all.htm#crc.cat.crc-32-iso-hdlc`
[4] `github.com/Henriquelay/polka_probe_checker`
[5] `github.com/mrhooray/crc-rs`

The solution has been tested against some adversarial scenarios, and checked against the same initial seed but under the base topology as described on 2.

4.1. Addition

An attacker PolKA switch s_{555} was added between s_5 and s_6, as shown in **??**. The packet was sent from e_1 to e_{10}. Suppose the attacking switch is properly connected in the ports that PolKA uses for this route, the packet is properly routed with PolKA. The checksums will not match when validating the path past the attacker. The packet trace is shown in **??**. Note the error propagating nature of composing checksums.

Comment

FIGURA

4.2. Detour

An attacker tries to make a detour in the network, as shown in **??**. The packet was sent from e_1 to e_{10}, and passed through the detour, as shown in **??**. The checksum will fail to validate when checking in the futures, as the function composition $f_{s_{555}}(x) = f_{s_6}(x)$. Note that this is only true when $k_{s_{555}} \neq k_{s_6}$. As stated, the key must be unique per node, and so must be kept secret.

Comment

FIGURA

4.3. Skipping

Misconfigured links can cause packets to skip nodes, as shown in **??**. The packet was sent from e_1 to e_{10}, and passed through the skipping, as shown in **??**. The checksum will not match when validating the path in the future, as the packet did not pass through the expected nodes.

Comment

FIGURA

4.4. Out-of-order TODO

4.5. Discussion

] - limitações - recuperar dados do mininet - sair dados nativamente do switch - cenários que não resolvemos (replay, etc) - limite de escala do cryptoscheme – é maior ou menor q o limite do Polka - tempo de cálculo do crypto line rate – impossível de maneira realista no nosso setup atual (mininet) - dificuldade de programar e debugar (wireshark) em p4

Upon developing the solution, a set of limitations were identified:

- As with most cryptographic solutions, the system is only as secure as the key used. If the key is compromised, the entire system is compromised, since a malicious actor can easily generate the same checksums.
- Replay attack is undetectable if metadata is disconsidered. This is due to the switch entry port not being included in the validation, which allows an attacker to replay the packet from a different port.

5. Conclusion and Future Work

The plan, as per the repository name implies, is to implement a non-reversible hash function, SipHash, more specifically, HalfSipHash[Aumasson and Bernstein 2012], to be used instead of the CRC32. This would make the system more secure, since CRC32 is a well-known as a checksum function that can be easily reversed[Stigge et al. 2006]. Also, a proper data compression method for adding (better term needed?) exit port and `node_id` into the checksum field is needed, since the current method is not optimal due to data loss from collision.

In the future, it should be integrated into PathSec[Martinello et al. 2024], and to do so the ingress edge needs to report the generated `key`, and the egress edge will report the final checksum directly to a blockchain, for auditability and accessibility. Having it directly report to a blockchain instead of a third party circumvents trust issues.

An interesting work can be done to use some sort of rotating key architecture to detect replay attacks. This is a difficult problem to solve, since the key must be rotated in a way that the attacker cannot predict, and the key must be shared between the nodes in a secure, atomic way to prevent the network to enter an irrecoverable state.

Including the entry port in the checksum would also be an appreciable increase in security, since it increases the number of targets an attacker would need to breach at the same time to be able to alter the path.

A timing analysis and stress tests can both be done to check if the incurred overhead is acceptable for the network. This is important, since the network must be able to handle the increased load without dropping packets. A more robust solution would be to use a more secure hash function, such as SHA-256, but this would increase the overhead of the network, and would require a more powerful hardware to be able to handle the increased load.

6. Conclusion

As the examples show, our solution is able to detect when a packet is not following the expected path for most cases, and can be used to detect misconfigurations in the network. The solution is not perfect, but it is a step in the right direction for a more secure network.

References

[bmv] Behavioral model. Accessed: 2024-09-26.

[Aumasson and Bernstein 2012] Aumasson, J.-P. and Bernstein, D. J. (2012). Siphash: a fast short-input prf. Cryptology ePrint Archive, Paper 2012/351.

[Basat 2020] Basat, B. (2020). Pint: Probabilistic in-band network telemetry. SIGCOMM '20, page 662–680, New York, NY, USA. Association for Computing Machinery.

[Dominicini et al. 2020] Dominicini, C., Mafioletti, D., Locateli, A. C., Villaca, R., Martinello, M., Ribeiro, M., and Gorodnik, A. (2020). Polka: Polynomial key-based architecture for source routing in network fabrics. pages 326–334.

[Fontes et al. 2015] Fontes, R., Afzal, S., Brito, S., Santos, M., and Rothenberg, C. E. (2015). Mininet-wifi: Emulating software-defined wireless networks.

[Foundation 2006] Foundation, W. (2006). Wireshark. Accessed: 2024-09-26.

[Lantz et al. 2010] Lantz, B., Heller, B., and McKeown, N. (2010). A network in a laptop: rapid prototyping for software-defined networks.

[Martinello et al. 2024] Martinello, M., Gomes, R. L., Borges, E. S., Layber, H. C., Bonella, V. B., Dominicini, C. K., Guimarães, R., Ribeiro, M., and Barcellos, M. (2024). Path-sec: Path-aware secure routing with native path verification and auditability. *Article*.

[secdev 2003] secdev, B. P. (2003). Scapy. Accessed: 2024-09-26.

[Stigge et al. 2006] Stigge, M., Plötz, H., Müller, W., and Redlich, J.-P. (2006). Reversing crc–theory and practice.